Unity FPS | 개발 포트폴리오

C# · Unity · FSM / State Pattern · NavMeshAgent · Raycast · ScriptableObject

FPS 전투·적 AI·인벤토리·HP 회복 시스템 구현 및 연동

소스코드: https://github.com/youngchan-kim/fps_project

프로젝트 개요

Unity FPS 개인 프로젝트에서 적의 탐지·추적·공격, 플레이어 사격과 피격, 아이템 관리 및 의료품 회복 기능을 연결했다. 기능 구현 과정에서 발견한 상태 관리·재장전·회복 버그를 분석하고 수정했다.

주요 구현

영역	구현 범위
Enemy AI	Idle / Reconnaissance / Run / Attack / Die; 탐지 및 NavMesh 이동
Shooting	단발·연사, Raycast 명중, 피해 전달, 탄창·재장전
Inventory	ScriptableObject 아이템, Ground / 일반 / 총기 슬롯 연동
HP / Healing	HPBar 갱신, 코루틴 회복, 회복 중 피격 반영

검증 표기: Unity 플레이 결과는 개발자가 직접 확인한 범위이며, 전체 자동화 테스트 완료를 의미하지 않는다.

01 | 시스템 아키텍처

입력과 행동, 피해 전달, 체력 및 UI를 담당하는 컴포넌트를 구분했다.

시작	연결 경로	책임
플레이어 사격	Player → GunSystem → DamegeSystem → HP_System	발사 조건 → 명중 → 체력 감소
의료품 사용	UserInterface → InventorySlot → HP_System → HPBar	아이템 소모 → 회복 → 표시
적 AI	TargetCheck ↔ Enemy / EnemyFSM → GunSystem	탐지 → 상태 전환 → 이동·공격

Enemy FSM 상태 전환

Idle은 장시간 대기가 아니라 현재 타겟 탐지 결과를 평가하는 분기 상태다. 타겟을 탐지하면 Run으로, 탐지하지 못하면 Reconnaissance로 전환한다. Run에서 공격 거리에 진입하면 Attack으로 이동하며, 타겟 상실 또는 거리 이탈 시 Idle로 복귀한다. 사망 여부는 EnemyFSM에서 공통 검사한다.

설계 특징

EnemyFSM은 상태 전환을, Enemy는 실제 NavMesh 이동과 공격 행동을 담당한다. GunSystem은 사격·재장전을 처리하고 DamegeSystem은 HP_System에 피해 적용을 요청한다.

관련 문서: Project/architecture.md

02 | 문제 해결: Enemy FSM

기존 문제

Enemy.cs의 switch 기반 상태 관리에 전환 조건이 집중되어 상태별 행동 흐름을 파악하고 수정하기 어려웠다.

개선 방법

IEnemyState(Enter / Update / Exit) 인터페이스를 도입하고 IdleState, ReconnaissanceState, RunState, AttackState, DieState를 분리했다. EnemyFSM.ChangeState()가 기존 상태 Exit와 새 상태 Enter를 호출하도록 했다.

설계 결정

Attack에서 타겟을 놓치면 Run으로 직접 이동하지 않고 Idle에서 재탐지한다. 타겟이 보이면 추적, 보이지 않으면 마지막 확인 위치 기반 정찰을 선택한다.

검증 및 제한

개발자는 Unity에서 탐지·추적·공격·정찰 복귀 및 HP 감소·사망의 핵심 동작을 확인했다. 오브젝트 풀링과 적 HP UI는 완료 기능으로 주장하지 않는다.

[실제 캡처 삽입 예정] 적 탐지 / 추적 / 정찰 / 상태 전환 로그

03 | 문제 해결: 재장전

현상

인벤토리에 탄약이 없어 실제 재장전할 수 없는 상태에서도 재장전 애니메이션이 실행되었다.

원인

Player의 실제 재장전은 ReloadIsPossible() 조건을 확인했지만, PlayerMovement의 애니메이션 시작은 R키 입력만 검사하여 조건이 분리되어 있었다.

수정

재장전 애니메이션 시작에도 현재 총기의 ReloadIsPossible() 조건을 적용했다. GunSystem은 총기 속성의 magazineSize로 탄창을 초기화하고, 인벤토리 보유 탄약과 부족한 탄약을 비교해 재장전한다.

검증 및 후속 과제

개발자는 탄약 소모 후 발사 제한, 인벤토리 탄약 재장전, 탄약이 없을 때 재장전 모션 제한을 확인했다. 재장전 중 무기 교체 및 적 전용 탄약 정책은 추가 검증이 필요하다.

[실제 캡처 삽입 예정] 탄약 부족 시 R키 / 정상 재장전 / 탄약 수량 변화

04 | 문제 해결: HP 회복

현상과 원인

회복 코루틴의 초기 버전에서는 CurrentHp가 실제 증가하지 않았다. 이후 고정 목표 HP 방식으로 수정하면서 회복 중 피격 피해가 최종 체력에 반영되지 않는 문제가 발생했다.

해결 방식

회복 시작 시 목표 HP를 고정하지 않고 remainingHeal(남은 회복량)을 관리한다. 매 프레임 회복 가능한 양을 CurrentHp에 더하고 동일한 양을 remainingHeal에서 차감한다. 회복 중 피해는 CurrentHp에 별도로 반영된다.

재현 및 검증

테스트	예상 최종 HP	개발자 확인
HP 30 + 회복 70	100	확인
HP 30 + 회복 70 + 중간 피해 10	90	확인
의료품 마지막 1개 사용	수량 감소 및 HP 증가	확인

[실제 캡처 삽입 예정] 회복 전 HP / 회복 중 피격 / 최종 HP / 의료품 수량

05 | 기여 범위와 다음 개선

참고한 구조

ScriptableObject 인벤토리의 기본 데이터, 슬롯, 드래그 UI는 공개 튜토리얼을 참고했다. 해당 기본 기능 전체를 독자 개발했다고 표현하지 않는다.

확장 및 개선

FPS 전용 Ground·총기 슬롯 처리와 의료품·HP 연동, Enemy FSM 상태 관리 리팩터링, 탄창·재장전 조건 연결 및 실제 버그 수정을 수행했다.

추가 검증이 필요한 항목

인벤토리 저장·복원, 정렬과 드래그 경계 사례, 총기 교체 중 재장전, 적 AI의 탄약 관리 분리, GameMgr·Player 초기화 순서 등.

증빙 자료 추가 후 최종본 제작

Images 폴더에 게임 실행 화면과 코드 캡처를 넣고, 각 페이지의 캡처 예정 문구를 실제 이미지로 교체한다. 이 문서는 이미지가 없는 텍스트 기반 제출용 초안이다.

상세 기술 문서: Project/overview.md · architecture.md · code-review.md · development-flow.md 및 3개 시스템 개발 문서